Towards a lightweight Spiking Neural P system for image recognition

Sandro Erba[1], Giuditta Franco[2], and Claudio Zandron[1]

[1] Dept. of Informatics, Systems and Communication, University of Milano-Bicocca, Milan, Italy
[2] Dept. of Computer Science, University of Verona, Verona, Italy

Abstract. In this paper, we investigate a variant of Spiking Neural P systems with discretized parameters at inference time, to propose a computational model capable of processing and classifying image data. A layered architecture is introduced, where input encoding, feature extraction, pooling, and classification are implemented through firing rules and synaptic connections. The model incorporates several design choices aimed at improving its expressiveness and stability, by including inhibitory synapses for handling negative interactions, forgetting rules that introduce non-linearity, and an extended rule encoding for compact representation. In addition, a biologically inspired neuron importance strategy is proposed, to modulate firing behavior, and an ensemble approach is used to combine multiple classifiers within the Spiking Neural P system framework. The system is evaluated on the MNIST dataset, by analyzing both the complete architecture and its single components. Experimental results highlight the trade-offs between model train phase size, performance, and memory usage.

Keywords: Biological Inspired Computing · Spiking Neural P systems · Image classification

1 Introduction

The paradigm of membrane computing, introduced in [17], has inspired a wide range of distributed and biologically motivated computational models. Among them, Spiking Neural P systems (abbreviated by SNPS) represent a notable class, which combines concepts from neural computation and formal language theory. Since their introduction, SNPS have been extensively studied from a theoretical perspective, with a strong focus on their computational power, universality, and formal properties [12]. Beyond their theoretical relevance, several extensions of SNPS have been proposed to enhance their learning capabilities and practical applicability. Different training strategies have been explored in the literature, including Hebbian-based mechanisms [11], optimization of probabilistic rule application [24], and structural plasticity approaches, where the network topology evolves during learning [2, 5, 18]. Models with weighted synapses [14] are extensions of the originally discrete SNPS, moving towards the traditional neural network paradigm for the sake of expressiveness. A related work [21] assumed a similar type of input, however

employing a separate P system with weights for each of the 26 English letters, and highlighting a different approach to multi-class processing.
Simulation tools have been developed to support experimentation with SNPS, by emphasizing both the growing interest in practical implementations and the challenges associated with scaling SNPS models. Namely, [9] presents a web-based simulator and [1] surveys multiple existing platforms by comparing their features.
A comprehensive overview of both the fundamentals and the variants of SNPS may be found in [25], whereas broader perspectives on membrane computing applied to image processing are discussed in [7]. More recent contributions have explored structured and layered architectures. For example, [8] proposes modular designs, and [26] investigates layered SNPS models for image recognition, including experiments on the MNIST dataset [6].
Image classification, and in particular handwritten digit recognition on MNIST, remains a standard benchmark for evaluating neural-inspired models. The dataset consists of grayscale images of size 28×28, belonging to 10 classes, with pixel intensities in the range $[0, 255]$. In machine learning, this task is typically addressed using Convolutional Neural Networks (CNNs), which exploit spatial locality and hierarchical feature extraction. A detailed introduction to CNN architectures and their principles may be found in [23], whereas practical design guidelines are discussed in [10].
This work aims at combining these two domains by investigating how core ideas of CNNs—such as feature extraction, pooling, and ReLU activation function—can be reformulated within the SNPS framework. The final inference model encodes the computation using discrete spikes, firing rules, and ternary synaptic connections, without real-valued synaptic weights. To support this objective, we report several extensions of the classical SNPS model present in the literature. More specifically, we consider: (*i*) an extended rule encoding that allows more flexible and compact representations of firing conditions; (*ii*) white hole rules, used to remove residual charges and stabilize the computation between two steps; and (*iii*) inhibitory synapses, allowing the generation of anti-spikes. All of these features are integrated into a deterministic framework.
To enable experimentation, we also developed a dedicated simulator capable of handling large-scale SNPS with the proposed extensions and supporting different configurations of the model.
In the next two sections, we introduce the proposed extended model, along with the layered structure of the network architecture (by detailing the role of rules and synapses), whereas in the last two sections, the main computational results are reported, for both the complete model and several sub-configurations, with a discussion about future research directions.

2 Extended model structure

Here we consider the **extended** SN P system, in which firing rules are allowed to emit more than one single spike. This variant was originally

introduced in [12, 16] and later formalized with this terminology in [3]. It is also discussed in more recent works such as [25, 13]. Emitting multiple spikes is an expressive encoding of information, particularly useful to process image data, where different spike multiplicities can be associated with variations in pixel intensity.

A second mechanism is the **white hole** feature. In this setting, whenever a neuron fires, all spikes currently stored in that neuron are consumed. The white hole mechanism was introduced in [19] and later referenced in [4]. Its inclusion is motivated by the need to process one image at each computational step, avoiding interference between consecutive inputs.

The model also incorporates **inhibitory synapses**, which allow certain connections to reduce the number of spikes in the target neuron. This concept was first introduced in [15] and further investigated in [20], with a formal definition provided in [22]. From a biological perspective, inhibitory synapses are essential for regulating neural activity. In this framework, spikes traversing inhibitory synapses are transformed into anti-spikes. A spike and an anti-spike annihilate each other upon interaction, ensuring that they cannot coexist within the same neuron. This annihilation process is applied immediately and in a maximal manner, as described in [15].

A distinguishing feature of this work is the absence of real-valued synaptic weights, as synaptic connections are not associated with continuous-valued parameters, unlike in many state-of-the-art neural models. This choice is motivated by the intention to preserve the original computational paradigm of P systems as much as possible. While this constraint inevitably leads to some performance degradation, the objective is to minimize such loss while maintaining a high degree of model fidelity.

Based on the extensions and assumptions described above, it follows our formal definition of the SN P system:

Spiking Neural P System with extended rules, white-hole neurons and inhibitory synapses

A spiking neural P system with extended rules, white-hole neurons and inhibitory synapses of degree $m \geq 1$ is a construct of the form

$$\Pi = (O, \sigma_1, \sigma_2, \ldots, \sigma_m, syn, in, out)$$

where:

- $O = \{a, \bar{a}\}$ is the alphabet; a denotes a spike and $\bar{a}$ and anti-spike.
- $\sigma_1, \sigma_2, \ldots, \sigma_m$ are neurons of the form $\sigma_i = (n_i, R_i)$, with $1 \leq i \leq m$, where $n_i \geq 0$ is the initial number of spikes contained in neuron σ_i and R_i is a finite set of rules of the following forms:
  - *Extended white-hole spiking rule*: $E/b^{all} \rightarrow b^p; d$, where E is a regular expression over a or $\bar{a}$, $b \in \{a, \bar{a}\}$, all denotes the total number of spikes or anti-spikes currently present in the neuron, where $all \geq p \geq 1$, and $d \geq 0$. If the neuron contains k spikes or anti-spikes b such that $b^k \in L(E)$, then all are consumed, and p spikes or anti-spikes are emitted through the outgoing synapses after d time units.

  - *White-hole forgetting rule*: $b^{all} \to \lambda$, with $b \in \{a, \bar{a}\}$, which is applied when $b^{all} \notin L(E)$ for any spiking rule. In this case all spikes are removed from the neuron.
- $syn = syn_e \cup syn_i \subseteq \{1, 2, \ldots, m\} \times \{1, 2, \ldots, m\}$, with $(i, i) \notin syn$, is the set of synapses between neurons, where:
  - syn_e is the set of excitatory synapses,
  - syn_i is the set of inhibitory synapses.

  Spikes transmitted through excitatory synapses are received normally by the target neuron, while spikes transmitted through inhibitory synapses are interpreted as anti-spikes.
- $in, out \subseteq \{1, 2, \ldots, m\}$ denote the sets of input and output neurons.

Rules codification and application

The model described in this paper has been implemented in a custom-built simulator[3] with a user-friendly GUI. Here, each neuron is characterized by a set of attributes, namely *id*, *initialCharge*, *outputTargets*, *neuronType*, and *rules*. The number of rules per neuron is not bounded a priori, and each rule is represented as a tuple of the form:

$$[div, mod, source, target, delay].$$

The pair (div, mod) encodes the regular expression that must be satisfied by the number of spikes contained in the neuron, *target* is the number of spike fired, *source* is the number of spike consumed when the rule applies, and *delay* is the refractory period for the neuron after the rules apply. In a more formal way, the rule can be rewritten as

$$a^{mod}(a^{div})^*/a^{source} \to a^{target}; delay.$$

Compact encoding via priority Rules are evaluated sequentially and applied according to their order of definition. When multiple rules are applicable, only the first one is selected. While this may appear as a priority-based resolution mechanism, it is in fact a compact encoding of an equivalent deterministic formulation.
To illustrate this, consider a neuron whose charge ranges from 1 to 255, and which must emit either 1 or 2 spikes depending on whether the charge exceeds half its maximum value (e.g., 128). In the proposed encoding, this behavior can be represented using only two rules:

$$[1, 128, 128, 2, 0], \quad [1, 1, 1, 1, 0],$$

that are equivalent to the regular expressions of the form

$$a^{128}(a)^*/a^{128} \to a^2; 0, \quad a(a)^*/a \to a; 0$$

where the first rule applies whenever the neuron contains at least 128 spikes and emits 2 spikes, while the second rule covers all remaining cases (except for $charge = 0$), emitting 1 spike. However, this compact

[3] *P system simulator*, GitHub repository, Available at: `https://github.com/SandroErba/spiking-p-system`.

representation is equivalent to an expanded formulation consisting of 255 rules with mutually exclusive applicability conditions:

$$[0, 255, 255, 2, 0],\ [0, 254, 254, 2, 0],\ \ldots,\ [0, 2, 2, 1, 0],\ [0, 1, 1, 1, 0],$$

which can be expressed as regular expressions of the form

$$a^{255} \to a^2, \quad a^{254} \to a^2, \quad \ldots, \quad a^2 \to a^1, \quad a \to a.$$

In this expanded formulation, each rule corresponds to an exact number of spikes, and for any given neuron charge, exactly one rule is applicable, deterministically. Let us remark that this type of rule transformation can only be applied if the maximum number of spikes that a neuron can contain is known, as it is precisely the case for our model.

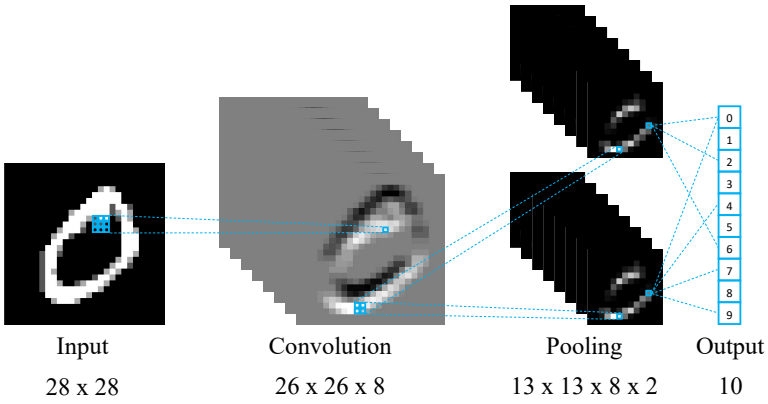


Fig. 1: Layers, with their size, composing the SNP system.

3 Network architecture

The proposed network architecture is structured into four layers, each integrating firing rules and synaptic connections to process image inputs within the Spiking Neural P system framework. An image representing the structure of the network can be seen in Figure 1. In each layer, the firing rules implement a specific feature, whilst the synapses might implement a different behaviour. The architecture is inherently scalable and can accommodate inputs of arbitrary size; however, for clarity and experimental consistency, the MNIST dataset is adopted as a reference, with its standard dimensions.

Layer 1: Input encoding and feature extraction The input image is mapped neuron by neuron, with each neuron corresponding to a distinct pixel. The size is therefore of $28 \cdot 28 = 784$ neurons. Firing rules convert

pixel intensities (ranging from 0 to 255) into a proportional number of spikes, depending on the chosen quantization range q. For example, with $q = 8$, the firing rules are represented as:

$$[1, 224, 224, 8, 0],\ [1, 192, 192, 7, 0], \ldots, [1, 32, 32, 2, 0],\ [1, 1, 1, 1, 0],$$

where the first rule fires if the neuron contains at least 224 spikes, the second if 192 or more, and so on. This representation is equivalent to a fully expanded set of 255 deterministic rules, ensuring that for each neuron charge, exactly one rule is applicable.

Neurons in Layer 1 project to Layer 2 through a set of eight fixed convolutional kernels, each designed to detect a specific edge orientation in the input image. The kernels correspond to the eight principal directions and are inspired by classical edge detection operators [18].

Each kernel is defined as a 3×3 matrix whose entries belong to the set $\{-1, 0, 1\}$, with an equal distribution of values. These values determine the type of synapse between neurons: excitatory (1), inhibitory (-1), or absent (0), and are applied with a stride of 1. An illustrative example of the kernel configuration and the transformation induced is provided in Figure 2. The background appears gray because the images contain both white points, corresponding to positive charges, and black points, corresponding to negative charges. In the visual representation, zero-charge values are mapped to an intermediate intensity, hence gray.

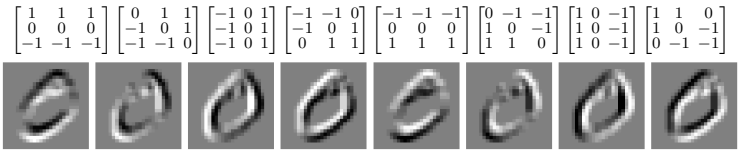


Fig. 2: Kernels defining the synapses configuration of Layer 1, and the resulting charge in Layer 2.

Layer 2: Convolution and forgetting mechanism Layer 2 is made of the eight feature maps from Layer 1, and is composed of $(28-2)\cdot(28-2)\cdot 8 = 5408$ neurons. Each neuron is equipped with firing rules that simply emit all the spikes currently stored, i.e., the output is equal to the neuron charge.

Neurons propagate their current charge while potentially accumulating negative charges due to inhibitory synapses coming from the Layer 1. To prevent residual negative charges from corrupting subsequent inputs, a forgetting rule is applied: neurons with *charge* < 0 reset to zero according to [-1,-1,0,0,0], which can be expressed as regular expressions of the form

$$\bar{a}(\bar{a})^* \to \lambda$$

This mechanism introduces a non-linear behavior in the network. Indeed, neurons with positive charge propagate their signal unchanged, while

neurons with non-positive charge are reset, by effectively discarding negative activations. This behavior is analogous to the Rectified Linear Unit (ReLU) activation function, commonly used after convolutional layers, thus enhancing an expressive power of the model despite the simplicity of its firing rules.
Each neuron in Layer 2 has exactly one outgoing synapse, which is always excitatory. Neurons are organized in groups of four, such that each group projects to a single neuron in Layer 3. Consequently, every neuron in Layer 3 receives input from exactly four neurons in Layer 2. This connectivity pattern implements an average pooling operation over non-overlapping 2×2 regions of the feature maps. By aggregating the contributions of four neighboring neurons into a single target neuron, the spatial resolution of each feature map is reduced by a factor of four. An example of accumulated charges in Layer 3, resulting from the aggregation of neurons from Layer 2, is shown in Figure 3. In this layer, the background of generated images appears black because the minimum value is 0, as negative values have been removed by the forgetting rule at Layer 2.



Fig. 3: Charge of the 8 kernels in Layer 3.

Layer 3: Average pooling, neuron importance, and training An average pooling block in Layer 3 consists of $13 \cdot 13 \cdot 8 = 1352$ neurons, as each neuron is formed by aggregating 4 neurons from the previous layer. The firing rules in Layer 3 are designed to implement an average pooling operation over the incoming spikes. Each neuron emits approximately one fourth of its internal charge, effectively reducing the accumulated input from Layer 2. Similarly to Layer 1, this behavior is encoded through a compact set of rules of the form:

$$[1, 96, 96, 24, 0],\ [1, 92, 92, 23, 0],\ \ldots,\ [1, 4, 4, 1, 0].$$

The upper bound of 96 spikes arises from the structure of the previous layers. In Layer 1, each neuron can emit at most 8 spikes (for $q = 8$). Due to the synaptic kernels, a neuron in Layer 2 can accumulate up to 24 spikes only in the ideal case where all excitatory synapses contribute their maximum value (8 spikes each), while inhibitory synapses do not generate anti-spikes. Since each neuron in Layer 3 receives input from exactly four neurons in Layer 2, as defined by the pooling structure, the maximum possible charge is $4 \cdot 24 = 96$ spikes.
Therefore, the firing rules map the accumulated charge to a reduced output by emitting a proportional number of spikes, specifically computing a discrete average over the inputs. This mechanism implements average

pooling directly through the rule dynamics, without requiring explicit division operations.

Layer 3 is directly connected to the final classification layer, and its synaptic connections define the decision process of the network. Unlike the previous layers, which are entirely rule-based and fixed, this layer represents the only component of the architecture where a training phase is performed.

Differently than some approaches existing in the literature for training Spiking Neural P systems, in this work we deliberately avoid the introduction of real-valued synaptic weights, preserving instead the ternary structure for synapses, with values in $\{-1, 0, 1\}$. The choice ensures compatibility with the discrete and symbolic nature of the model, while maintaining computational efficiency.

The training procedure is performed as in the following. For each input sample, the internal charges of the 1352 neurons in Layer 3 are collected, resulting in a feature representation of the input. By aggregating all training samples, a matrix of size $1352 \times N$ is obtained, where N is the size of the training set. The matrix is used as input for two standard machine learning classifiers, namely a Support Vector Machine (SVM) and a Logistic Regression (LogReg) model. Both models learn to map the features extracted by the first three layers of the SNPS into the corresponding class labels. As a result of the training process, each model produces a real weight matrix of size 1352×10, corresponding to the contribution of each neuron to each output class. Within the matrix, an *importance score* is assigned to each neuron of Layer 3, to quantify the contribution of each neuron to the classification task, by distinguishing between neurons that carry discriminative information and those that mainly introduce noise during the firing process.

A key idea defines the concept of neuron importance. For each neuron, the normalized difference between the maximum and minimum weight across the 10 output classes is computed, by providing us with a measure in the range $[0, 1]$ of how heavily the neuron contributes to class discrimination.

The resulting score is subsequently discretized into a user-defined range. In practice, a range of 6 levels has proven to be effective, assigning to each neuron an importance value in $\{1, 2, \ldots, 6\}$. This value is used to modulate the firing rules of the neuron: instead of emitting a fixed fraction of spikes as defined by the average pooling rules, the number of emitted spikes is scaled proportionally to the assigned importance.

As a consequence, all firing rules associated with the neuron are uniformly adjusted, effectively introducing a rule tuning mechanism within the Spiking Neural P system. For example, with a value 4 of importance, the new rules will become

$$[1, 96, 96, 96, 0],\ [1, 92, 92, 92, 0],\ \ldots,\ [1, 4, 4, 4, 0].$$

This approach allows the model to retain part of the information encoded in the original real-valued weights, and also introduces a potential risk. The external classifiers are trained on features obtained after average pooling, where each neuron emits one fourth of its charge. Scaling these values through the importance factor may alter the distribution

of the features, potentially degrading the effectiveness of the learned representations. The impact of this mechanism is therefore evaluated in section 4. The real-valued weight matrix obtained from the pretrained classifier must then be converted into ternary weights in order to integrate it into the Spiking Neural P system framework. This conversion is performed using a percentile-based method, with thresholds defined by the user. Specifically, the highest weights above the positive percentile are mapped to $+1$, the lowest weights below the negative percentile are mapped to -1, and the intermediate weights are set to 0. The impact of these percentile thresholds on network performance will be analyzed in section 4.

Layer 3: Ensemble As it can be seen in Figure 1, Layer 3 does not contain exactly 1352 neurons. Since two classifiers are employed—SVM and LogReg—then two separate weight matrices are generated, by yielding two importance scores for each neuron. To preserve all discriminative information provided by both classifiers, the network maintains both sets of neurons, effectively doubling the number of neurons in Layer 3. Consequently, each neuron of Layer 2 now projects into two corresponding neurons in Layer 3: one associated with the SVM classifier and the other one with the Logistic Regression classifier.
It is worth noting that if the network is used as an ensemble without applying the importance scores, then doubling the neurons is unnecessary: in this case, each neuron in Layer 2 can project into a single Layer 3 neuron with two separate synaptic connections, derived from the two classifiers' weight matrices. Without the importance scaling, the neurons of Layer 3 simply fire one quarter of their accumulated spikes, by yielding equivalent behavior.

Layer 4: Classification Layer 4 performs the final classification, by integrating both the importance-scaled outputs in Layer 3 and the ensemble effect from multiple classifiers. Neurons (one per class) accumulate incoming spikes and fire according to their internal charge, while the neuron emitting the highest number of spikes determines the predicted class.

4 Results

An initial phase of parameter exploration was conducted in order to identify a suitable operating configuration. Unless explicitly varied in their corresponding plots, the parameters are fixed as it follows: the *q-range* is set to 11, the training set size to 5000 samples, the sparsity level to 0.8, and the fraction of positive connections to 0.1. All subsequent analyses are performed by varying one parameter at a time while keeping the others fixed at these values.
As for our experimental setup, we analyze classification accuracy across four key factors: *q-range*, *training set size*, *sparsity*, and *positive ratio*.
Each experiment is summarized through a dedicated plot in Figure 4, showing how performance evolves as a function of one parameter while

keeping the others controlled. Each plot reports the performance of ten different configurations, ordered as follows:

1. Support Vector Machine (SVM) and Logistic Regression (LogReg) applied directly to the input
2. SVM and LogReg ternarized and applied on Layer 3 of the SNPS
3. SVM and LogReg ternarized and applied on Layer 3 of the SNPS enhanced with importance weighting
4. Ensemble model combining SVM and LogReg ternarized and applied on Layer 3 of the SNPS, also enhanced with importance weighting
5. SVM and LogReg applied to the third layer with real-valued weights

The goal of the evaluation is to compare the effectiveness of SNPS-based representations, their enhanced variants, and standard baselines in terms of classification accuracy under varying conditions. For a visualization, we adopt a consistent encoding scheme across all plots:

- **Circles** represent SVM-based models
- **Squares** represent LogReg-based models
- **Plus markers (+)** represent ensemble models
- **Dashed lines** represent non SNPS models (SVM and LogReg when applied externally with respect to the network)

This unified representation allows for immediate visual comparison between model families, highlighting the contribution of feature extraction of SNPS, importance weighting, and ensemble strategies.

Impact of Training size Figure 4(a) shows the effect of increasing the training set size on the classification accuracy of all considered models. A first notable observation concerns the behavior of SNPS representations when combined with different classifiers. In particular, the SNPS + SVM configuration exhibits a clear degradation in performance as the training size increases, suggesting a tendency to overfit or to become unstable when exposed to larger datasets. In contrast, the SNPS + Logistic Regression model remains significantly more stable, maintaining relatively consistent performance across all training sizes.
On the other hand, models using real-valued weights (i.e., non-SNPS variants) show a steady, albeit moderate, improvement as the number of training samples increases. This behavior is expected, as the models can better exploit the additional data to refine their decision boundaries. A key result is that SNPS-based models, especially the variants incorporating importance weighting and ensemble strategies, reach near-optimal performance with relatively small training sizes (around 6k samples). Beyond this point, additional data can provide only marginal gains, indicating that the SNPS representation may be considered data-efficient.
These results highlight a trade-off between data efficiency and scalability: SNPS-based models achieve the best performance with limited data, whereas models based on real-valued weights continue to benefit from larger training sets.

Impact of Quantization range Figure 4(b) reports the effect of the quantization range on classification accuracy. This parameter controls the number of discrete levels (spikes) used to represent the input, thus directly affecting the amount of information retained after quantization.

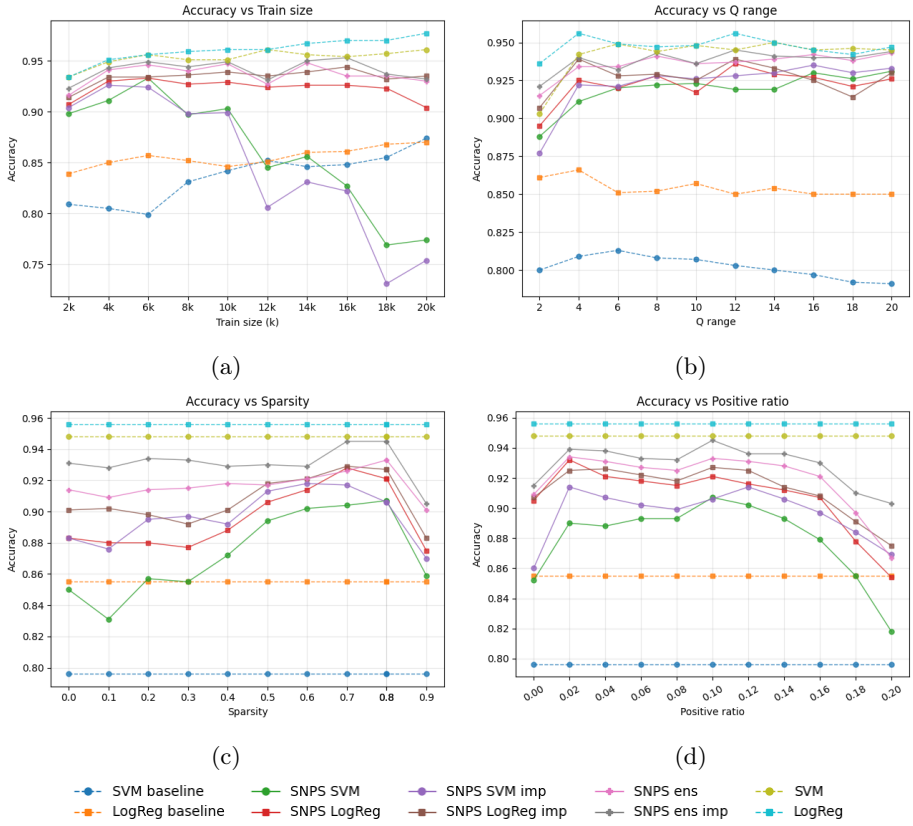


Fig. 4: Accuracy varying by changing the train size (a), the quantization range (b), the sparsity (c), and the ratio of positive synapses (d).

As expected, extremely low values of q-range (e.g., *q-range*= 2) lead to a notable degradation in performance across all models, due to the severe loss of information in the input representation. However, already at *q-range*= 4, all configurations exhibit a significant recovery, reaching performance levels close to their maximum.

Beyond this point, increasing the *q-range* produces only marginal improvements. The results indicate that the proposed approach is robust to aggressive quantization and can capture the relevant information for classification even under strong information constraints.

Impact of Sparsity Figure 4(c) shows the effect of increasing sparsity, defined as *the percentage of weights set to zero during the three-discretization process*. The remaining connections are equally divided between positive and negative values.

A clear and consistent trend emerges across all SNPS-based models: performance improves as sparsity increases, up to a certain threshold.
The above behavior suggests that a fully connected layer introduces a significant amount of redundancy and noise, and that removing less informative connections leads to improved generalization. This effect is reminiscent of synaptic pruning observed in biological neural systems, where redundant connections are progressively eliminated to enhance efficiency and functionality. However, for very high sparsity values (e.g., 0.9), performance drops sharply across all models, indicating that excessive pruning removes not only redundant but also informative connections. At lower sparsity levels, clear differences between models are observed. Logistic Regression consistently outperforms SVM, while ensemble models further improve upon both. The contribution of importance weighting is consistently positive, yielding improvements of approximately 1–2% in the low-to-mid sparsity regime. However, its impact diminishes at higher sparsity levels, where the only pruning process is sufficient to remove most of the irrelevant connections.
As expected, models based on real-valued weights are unaffected by sparsity, resulting in flat performance curves. Overall, these results indicate that sparsity acts as an effective regularization mechanism, improving both efficiency and accuracy when applied at an appropriate level.

Impact of Positive/Negative Balance Figure 4(d) analyzes the effect of varying the proportion of positive connections while keeping the overall sparsity fixed at 0.8, which corresponds to the optimal regime identified in the previous experiment. Increasing the fraction of positive weights necessarily reduces the number of negative ones, and vice-versa.
The results show that the best performance is achieved when positive and negative connections are approximately balanced. Moving towards either extreme, where one type of connection dominates, leads to a consistent degradation in accuracy. This indicates that both positive and negative contributions carry information that is necessary for effective classification.
Importance weighting consistently provides a modest but reliable improvement across all configurations. Ensemble models remain among the top-performing approaches, confirming their robustness across different parameter regimes. Overall, these results highlight that maintaining a balance between positive and negative connections is crucial and that overly skewed representations lead to a loss of discriminative power.

5 Discussion

The experimental results consistently point to a regime in which performance is not limited by the amount of raw information available, rather by how that information is structured and filtered. Across all experiments, increasing the resolution of the input representation or the amount of training data yields only marginal improvements beyond relatively low thresholds. In contrast, structural modifications of the representation—such as sparsification and balancing of positive and negative connections—have a significantly stronger impact on performance.

In particular, sparsity emerges as the dominant factor: removing a large fraction of connections improves accuracy, indicating that dense representations introduce redundancy and noise rather than useful information. However, this effect is not unbounded, and excessive sparsity leads to a rapid loss of performance, highlighting the existence of an optimal compression regime.
At the same time, the balance between positive and negative contributions proves to be essential. The best results are consistently achieved when the two are approximately balanced, suggesting that the discriminative signal is distributed across both components and that skewed representations lead to information loss.
Another key observation is that SNPS-based models reach their optimal performance with limited data and low quantization levels, indicating high data efficiency. However, this also implies limited scalability, as additional data or increased precision does not translate into significant gains. Finally, importance weighting and ensemble strategies provide moderate however consistent improvements, acting as refinement mechanisms rather than fundamentally altering model behavior.
While the proposed approach achieves competitive performance on the MNIST dataset, a significant degradation is observed when applied to more complex datasets. This behavior can be attributed to the nature of the feature extraction process. In the case of MNIST, the use of simple edge-detection kernels is sufficient to extract discriminative features, as the task mainly relies on basic shape information.
However, for more complex visual tasks, a shallow feature extraction strategy is not sufficient. These problems typically require multiple layers of feature extraction, to capture higher-level patterns, textures, and hierarchical structures within the data. Since the current approach relies on a limited transformation stage, it is unable to fully represent more complex characteristics, leading to a drop in performance.
The above limitation is further confirmed by preliminary experiments, conducted on more challenging medical imaging datasets from the MedMNIST benchmark, where the obtained results are not comparable to those achieved on the state of the art. These findings suggest that extending the method with deeper or more expressive feature extraction mechanisms is a necessary direction for future work.

Memory footprint

A relevant strength of our work is the memory occupancy of the network. Only the top layer needs to be stored, as the rest of the structure is fixed and clearly defined, and identical for all the models released. Therefore, we are going to focus solely on the neurons in the third layer and their connections. Let:

- c = number of output classes (synapses per neuron)
- s = sparsity level (fraction of deleted synapses)
- r = range for the importance (scaling of the third layer)

Each synaptic weight is ternary, and each neuron has a bias (importance value). The space required for a single bias is $\lceil \log_2 r \rceil$, and for a single ternary weight it is $\lceil \log_2 3 \rceil$. We now calculate the memory requirements

for storing a single neuron, and then apply them to the studied network to provide a practical example.
Further two points to remark are the density and the sparsity of encoding. Indeed, the memory cost for every neuron is $N_{\text{dense}} = \lceil \log_2 3 \rceil \cdot c + \lceil \log_2 r \rceil = 2c + \lceil \log_2 r \rceil$ bits. Moreover, in the sparse configuration, the number of non–zero synapses in every neuron is $(1 - s) \cdot c$, and each synapse requires $\lceil \log_2 c \rceil$ bits for the class index and 1 bit for the sign (spike or anti-spike). Therefore, the memory cost becomes $N_{\text{sparse}} = (1 - s) \cdot c \cdot (\lceil \log_2 c \rceil + 1) + \lceil \log_2 r \rceil$.
Sparse encoding becomes advantageous when $N_{\text{sparse}} < N_{\text{dense}}$, which leads to a *sparsity threshold* $s^* = 1 - \frac{2}{\lceil \log_2 c \rceil + 1}$. For $s > s^*$, the sparse encoding requires less memory than the dense encoding.
Specifically, for the benchmark MNIST, we consider a model with $c = 10$ output classes, a sparsity level of $s = 0.8$, and an importance range of $r = 6$, consistent with the experimental setup. We therefore obtain $\lceil \log_2 c \rceil = 4$ e $\lceil \log_2 r \rceil = 3$. We obtain a dense encoding by substituting the model parameters in the previous equations: $N_{\text{dense}} = 2 \cdot 10 + 3 = 23$ bits for every neuron.
Regarding the sparse encoding: with 10 classes the sparsity value s above which this approach becomes advantageous is $s^* = 1 - \frac{2}{4+1} = 0.6$. With sparsity $s = 0.8$, we have $N_{\text{sparse}} = 0.2 \cdot 10 \cdot (4 + 1) + 3 = 13$ bits per neuron.
In the case of the ensemble model with importance, which is the heaviest and highest-performing, the total model size is calculated based on the number of neurons n, which is $n = 1352 \cdot 2 = 2704$. Therefore, the model requires 62,192 bits (approximately 7.59 KB) using the dense method, and 35,152 bits (approximately 4.29 KB) using the sparse method.

In conclusion, this work demonstrates that SNPS may be effectively adapted to structured tasks such as image classification, while preserving their discrete and biologically inspired nature. Inhibitory synapses and anti-spikes were exploited to enrich neuron interactions, while the introduction of one forgetting rule enabled a behavior analogous to ReLU activation operators, by introducing non-linearity. The adoption of the white hole rule mechanism allowed the system to process a new input at each time step, by making the model suitable for sequential data processing. A detailed analysis of the required memory was finally carried out, by considering both rules and synaptic connections, and providing insights into the scalability of the approach.
The proposed extensions open up new directions for integrating learning mechanisms and efficient computation within the membrane computing framework.
Future work will focus on developing an integrated training procedure directly within the proposed architecture, by enabling kernel optimization and by reducing the reliance on external classifiers. One of the main future objectives is to ensure that the model's performance remains competitive even for more complex datasets. Further investigation will try to deepen the impact of the importance mechanism and leverage the multiple uses of the firing rules.

Bibliography

[1] Cabarle, F.G.C.: Thinking about spiking neural P systems: some theories, tools, and research topics. Journal of Membrane Computing **6**(2), 148–167 (2024)

[2] Cabarle, F.G.C., Adorna, H.N., Pérez-Jiménez, M.J., Song, T.: Spiking neural P systems with structural plasticity. Neural Computing and Applications **26**(8), 1905–1917 (2015)

[3] Chen, H., Ionescu, M., Ishdorj, T.O., Păun, A., Păun, G., Pérez-Jiménez, M.J.: Spiking neural P systems with extended rules: Universality and languages. Natural Computing **7**(2), 147–166 (2008). https://doi.org/10.1007/s11047-006-9024-6

[4] Chen, Y., Chen, Y., Zhang, G., et al.: A survey of learning spiking neural P systems and a novel instance. International Journal of Unconventional Computing **16**(2-3), 173–200 (2021)

[5] de la Cruz, R.T.A., Cabarle, F.G.C., Macababayao, I.C.H., Adorna, H.N., Zeng, X.: Homogeneous spiking neural P systems with structural plasticity. Journal of Membrane Computing **3**(1), 10–21 (2021)

[6] Deng, L.: The mnist database of handwritten digit images for machine learning research. IEEE signal proc. magazine **29**(6), 141–142 (2012)

[7] Díaz-Pernil, D., Gutiérrez-Naranjo, M.A., Peng, H.: Membrane computing and image processing: a short survey. Journal of Membrane Computing **1**(1), 58–73 (2019)

[8] Ermini, I., Zandron, C.: Modular spiking neural membrane systems for image classification. International Journal of Neural Systems **34**(06), 2450021 (2024). https://doi.org/10.1142/S0129065724500217

[9] Gulapa, M., Luzada, J.S., Cabarle, F.G.C., et al.: Websnapse reloaded: The next-generation spiking neural P system visual simulator using. In: WCTP 2023: Theory and practice. p. 434. Springer (2024)

[10] Gurucharan, M.: Basic CNN Architecture Explained: Key Layers and Workflow. https://www.upgrad.com/blog/basic-cnn-architecture/ (2026)

[11] Gutiérrez-Naranjo, M.A., Pérez-Jiménez, M.J.: Hebbian learning from spiking neural P systems view. In: International workshop on membrane computing. pp. 217–230. Springer (2008)

[12] Ionescu, M., Păun, G., Yokomori, T.: Spiking neural P systems. Fundamenta Informaticae **71**(2-3), 279–308 (2006). https://doi.org/10.3233/FUN-2006-712-308

[13] Leporati, A., Mauri, G., Zandron, C.: Spiking neural P systems: Main ideas and results. Natural Computing **21**(4), 629–649 (2022). https://doi.org/10.1007/s11047-022-09917-y

[14] Nan, H., Chen, H., Guo, P., Liao, C., Tahmid, S.A.: Competitive nonlinear layered spiking neural P system for solving classification problems. Neurocomputing **637**, 130036 (2025)

[15] Pan, L., Păun, G.: Spiking Neural P Systems with Anti-Spikes. Int. J. of Computers Communications and Control **4**(3), 273–282 (2009)
[16] Păun, A., Păun, G.: Small universal spiking neural P systems. Biosystems **90**(1), 48–60 (Jul 2007). https://doi.org/10.1016/j.biosystems.2006.06.006
[17] Păun, G.: Computing with membranes. Journal of computer and system sciences **61**(1), 108–143 (2000)
[18] Robertson, J., Kirkland, P., Alanis, J.A., Hejda, M., Bueno, J., Di Caterina, G., Hurtado, A.: Ultrafast neuromorphic photonic image processing with a VCSEL neuron. Scientific Reports **12**(1), 4874 (2022). https://doi.org/10.1038/s41598-022-08703-1
[19] Song, T., Gong, F., Liu, X., Zhao, Y., Zhang, X.: Spiking neural p systems with white hole neurons. IEEE Transactions on NanoBioscience **15**(7), 666–673 (2016). https://doi.org/10.1109/TNB.2016.2598879
[20] Song, T., Pan, L., Wang, J., Venkat, I., Subramanian, K.G., Abdullah, R.: Normal forms of spiking neural P systems with anti-spikes. IEEE Transactions on NanoBioscience **11**(4), 352–359 (2012). https://doi.org/10.1109/TNB.2012.2208122
[21] Song, T., Pan, L., Wu, T., Zheng, P., Wong, M.D., Rodríguez-Patón, A.: Spiking neural p systems with learning functions. IEEE transactions on nanobioscience **18**(2), 176–190 (2019)
[22] Song, T., Wang, X.: Homogenous spiking neural P systems with inhibitory synapses. Neural Processing Letters **42**(1), 199–214 (2015). https://doi.org/10.1007/s11063-014-9352-y
[23] Stanković, L., Mandic, D.: Convolutional neural networks demystified: A matched filtering perspective-based tutorial. IEEE Transactions on Systems, Man, and Cybernetics: Systems **53**(6), 3614–3628 (2023). https://doi.org/10.1109/TSMC.2022.3228597
[24] Zhang, G., Rong, H., Neri, F., Pérez-Jiménez, M.J.: An optimization spiking neural P system for approximately solving combinatorial optimization problems. International Journal of Neural Systems **24**(05), 1440006 (2014)
[25] Zhang, G., Verlan, S., Wu, T., et al.: Fundamentals of Spiking Neural P Systems. In: Spiking Neural P Systems: Theory, Applications and Implementations, pp. 3–29. Springer Nature, Singapore (2024). https://doi.org/10.1007/978-981-97-9282-5_1
[26] Zhang, G., Zhang, X., Rong, H., Paul, P., Zhu, M., Neri, F., Ong, Y.S.: A layered spiking neural system for classification problems. International Journal of Neural Systems **32**(08), 2250023 (2022). https://doi.org/10.1142/S012906572250023X